

Übungseinheit: Funktionen und Datenstrukturen

Anwendung von Listen, Dictionaries und eigenen Funktionen

Dorian Zwanzig, Claude

12. November 2025

Inhaltsverzeichnis

1	Über diese Übung	3
1.1	Lernziele	3
1.2	Voraussetzungen	3
1.3	Hinweis	3
2	Vorbereitung	4
2.1	Technische Vorbereitung	4
2.2	Mentale Vorbereitung	4
2.3	Test der Umgebung	4
3	Einführung: Funktionen & Datenstrukturen	5
3.1	Warum Funktionen mit Listen und Dictionaries?	5
3.2	Grundprinzip: Funktion als Datenverarbeiter	5
3.3	Beispiel: Funktion mit Liste	5
3.4	Beispiel: Funktion mit Dictionary	5
4	Übungsblock A – Listen und Funktionen	6
4.1	Vorbereitung	6
4.2	Aufgabe 1: Durchschnitt berechnen	6
4.3	Aufgabe 2: Maximum finden	6
4.4	Aufgabe 3: Liste analysieren	7
5	Übungsblock B – Dictionaries und Funktionen	9
5.1	Aufgabe 4: Produktpreis berechnen	9
5.2	Aufgabe 5: Studenten-Dictionary erweitern	10
5.3	Aufgabe 6: Zwei Dictionaries vergleichen	11
6	Übungsblock C – Vertiefung Funktionskonzepte	12
6.1	Aufgabe 7: Lokale vs. Globale Variablen	12
6.2	Aufgabe 8: Variable Shadowing	13
6.3	Aufgabe 9: Scope-Kette	14
6.4	Aufgabe 10: Positional vs. Named Parameters	14
6.5	Aufgabe 11: Parameter-Reihenfolge verstehen	15
6.6	Aufgabe 12: Gemischte Parameter-Strategie	16
6.7	Aufgabe 13: Return vs. Print	17
6.8	Aufgabe 14: Mehrfache Return-Werte	18
6.9	Aufgabe 15: Return-Kette (Funktionen verketten)	19
7	Erweiterte Aufgabe – Kombination aller Konzepte	21
7.1	Komplexaufgabe: Lagerverwaltungssystem	21
8	Musterlösungen	25
8.1	Lösungen Block A – Listen und Funktionen	25

8.2	Lösungen Block B – Dictionaries und Funktionen	27
8.3	Lösungen Block C – Vertiefung Funktionskonzepte	28
8.4	Lösung Erweiterte Aufgabe	34
9	Typische Anfängerfehler – Troubleshooting	36
10	Abschluss & Reflexion (ca. 10 Min)	37
10.1	Was Sie gelernt haben	37
10.2	Reflexionsfragen	37
10.3	Ausblick	37
10.4	Selbsteinschätzung	37

1 Über diese Übung

1.1 Lernziele

Nach Abschluss dieser Übung können Sie: - Listen und Dictionaries sicher erstellen und verwenden - Eigene Funktionen definieren und aufrufen - Das Scope-Konzept verstehen und anwenden (lokal vs. global) - Positionale und benannte Parameter gezielt einsetzen - Return-Statement effektiv nutzen und von print() unterscheiden - Arithmetische und logische Operatoren anwenden - Funktionen zur Verarbeitung von Datenstrukturen schreiben - Mehrere Konzepte in praktischen Szenarien kombinieren

Zeitbedarf: ca. 90-180 Minuten **Schwierigkeitsgrad:** ** (Mittel)

1.2 Voraussetzungen

Für diese Übung sollten Sie folgende Konzepte sicher beherrschen: - **Variablen und einfache Datentypen** (Notebooks 04-05) - **Listen und Dictionaries** (Notebook 06) - **Funktionen definieren und aufrufen** (Notebook 07) - **Arithmetische und logische Operatoren** (Notebook 08)

Falls Sie unsicher sind, wiederholen Sie bitte die entsprechenden Notebooks.

1.3 Hinweis

Legen Sie mindestens für jeden Übungsblock eine eigene Datei an, idealerweise für jede Übung. Das hilft Ihnen die Übersicht zu behalten - nicht nur heute, sondern auch während der Klausurvorbereitung.

2 Vorbereitung

Aufwand ca. 10 Minuten.

2.1 Technische Vorbereitung

Erstellen Sie eine neue Python-Datei `uebung_funktionen_start.py` und öffnen Sie diese in Ihrer Entwicklungsumgebung.

2.2 Mentale Vorbereitung

In dieser Übung werden Sie: 1. **Listen und Dictionaries** als Container für Daten verwenden 2. **Eigene Funktionen** schreiben, die diese Daten verarbeiten 3. **Operatoren** verwenden, um Berechnungen und Vergleiche durchzuführen

Wichtig: Wir verwenden noch keine Schleifen oder if-Anweisungen. Stattdessen greifen wir mit Indizes auf einzelne Elemente zu.

2.3 Test der Umgebung

Führen Sie folgenden Code aus, um Ihre Umgebung zu testen:

```
# Umgebungstest
print("Python-Umgebung bereit!")

# Test: Liste erstellen und ausgeben
test_liste = [1, 2, 3]
print(f"Liste: {test_liste}")

# Test: Dictionary erstellen und ausgeben
test_dict = {"name": "Test"}
print(f"Dictionary: {test_dict}")

# Test: Einfache Funktion
def teste():
    return "Funktionen funktionieren!"

print(teste())
```

Erwartete Ausgabe:

```
Python-Umgebung bereit!
Liste: [1, 2, 3]
Dictionary: {'name': 'Test'}
Funktionen funktionieren!
```

3 Einführung: Funktionen & Datenstrukturen

Aufwand ca. 15 Minuten.

3.1 Warum Funktionen mit Listen und Dictionaries?

In der Praxis arbeiten Programme selten mit einzelnen Werten. Meist verarbeiten wir **Sammlungen von Daten**: - Eine Liste von Produkten mit ihren Preisen - Studentendaten mit Namen und Noten - Messwerte aus Experimenten

Funktionen helfen uns, diese Datenverarbeitung zu **strukturieren** und **wiederverwendbar** zu machen.

3.2 Grundprinzip: Funktion als Datenverarbeiter

```
def verarbeite_daten(eingabe_daten):  
    # Verarbeitung  
    ergebnis = # ... Berechnung ...  
    return ergebnis
```

3.3 Beispiel: Funktion mit Liste

```
def berechne_summe_erste_drei(zahlen_liste):  
    """Berechnet die Summe der ersten drei Elemente einer Liste"""  
    summe = zahlen_liste[0] + zahlen_liste[1] + zahlen_liste[2]  
    return summe  
  
# Verwendung  
meine_zahlen = [10, 20, 30, 40, 50]  
ergebnis = berechne_summe_erste_drei(meine_zahlen)  
print(f"Summe der ersten drei: {ergebnis}")
```

3.4 Beispiel: Funktion mit Dictionary

```
def erstelle_begrueessung(person_dict):  
    """Erstellt eine Begrüßung aus einem Personen-Dictionary"""  
    vollname = person_dict["vorname"] + " " + person_dict["nachname"]  
    begrueessung = f"Hallo {vollname}, willkommen!"  
    return begrueessung  
  
# Verwendung  
person = {"vorname": "Anna", "nachname": "Müller"}  
nachricht = erstelle_begrueessung(person)  
print(nachricht)
```

4 Übungsblock A – Listen und Funktionen

Aufwand ca. 20-30 Minuten.

Fortschritt: [## _____] 20% der Übung

4.1 Vorbereitung

Erstellen Sie eine neue Python Datei für die Übung.

Info-Box: Listen-Zugriff ohne Schleifen

Da wir noch keine Schleifen kennen, greifen wir mit **Indizes** auf Listenelemente zu: - `liste[0]` - erstes Element - `liste[1]` - zweites Element - `liste[-1]` - letztes Element - `len(liste)` - Anzahl der Elemente

4.2 Aufgabe 1: Durchschnitt berechnen

4.2.1 Aufgabenstellung

Schreiben Sie eine Funktion `berechne_durchschnitt_drei()`, die eine Liste mit genau drei Zahlen erhält und deren Durchschnitt berechnet.

Testen Sie mit der Liste `[15, 20, 25]`.

4.2.1.1 Hilfestellungen Problemanalyse

- Was ist der Durchschnitt? → Summe aller Werte geteilt durch die Anzahl
- Wie greift man auf Listenelemente zu? → Mit Index: `liste[0]`, `liste[1]`, `liste[2]`
- Was soll die Funktion zurückgeben? → Den berechneten Durchschnitt

Lösungsschritte

1. Funktion mit Parameter `zahlen` definieren
2. Auf die drei Elemente zugreifen: `zahlen[0]`, `zahlen[1]`, `zahlen[2]`
3. Summe berechnen durch Addition
4. Durch 3 teilen
5. Ergebnis mit `return` zurückgeben
6. Funktion aufrufen und Ergebnis ausgeben

Pseudocode

```
FUNKTION berechne_durchschnitt_drei(zahlen):
    summe = zahlen[0] + zahlen[1] + zahlen[2]
    durchschnitt = summe / 3
    RÜCKGABE durchschnitt

test_zahlen = [15, 20, 25]
ergebnis = berechne_durchschnitt_drei(test_zahlen)
AUSGABE f"Der Durchschnitt ist: {ergebnis}"
```

4.2.2 Erwartete Ausgabe

Der Durchschnitt ist: 20.0

4.3 Aufgabe 2: Maximum finden

Schwierigkeit: ** (Leicht-Mittel)

4.3.1 Aufgabenstellung

Schreiben Sie eine Funktion `finde_maximum_zwei()`, die das größte Element aus einer Liste mit zwei Zahlen zurückgibt.

Testen Sie mit `[45, 89]`.

4.3.1.1 Hilfestellungen Problemanalyse

- Wie vergleicht man zwei Werte? → Mit `>` Operator
- Was ergibt ein Vergleich? → `True` (= 1) oder `False` (= 0)
- Wie nutzt man Boolean in Berechnungen? → `True * zahl = zahl`, `False * zahl = 0`
- Wie wählt man das Maximum ohne `if`? → Multiplikation mit Boolean und Addition

Lösungsschritte

1. Funktion mit Parameter `zahlen` definieren
2. Beide Elemente in Variablen speichern: `a = zahlen[0]`, `b = zahlen[1]`
3. Vergleich durchführen: `a_groesser = a > b`
4. Trick anwenden: `maximum = a * a_groesser + b * (not a_groesser)`
5. Ergebnis mit `return` zurückgeben

Pseudocode

```
FUNKTION finde_maximum_zwei(zahlen):
    a = zahlen[0]
    b = zahlen[1]

    a_groesser_b = a > b      # True oder False
    b_groesser_a = b > a      # umgekehrt

    # True = 1, False = 0
    maximum = a * a_groesser_b + b * b_groesser_a

    RÜCKGABE maximum

test_zahlen = [45, 89]
ergebnis = finde_maximum_zwei(test_zahlen)
AUSGABE f"Das Maximum ist: {ergebnis}"
```

4.3.2 Erwartete Ausgabe

Das Maximum ist: 89

4.4 Aufgabe 3: Liste analysieren

Schwierigkeit: ** (Leicht-Mittel)

4.4.1 Aufgabenstellung

Schreiben Sie eine Funktion `analysiere_liste()`, die eine Liste mit vier Zahlen analysiert und einen String mit folgenden Informationen zurückgibt: - “anzahl”: Anzahl der Elemente (mit `len()`) - “erstes”: Erstes Element - “letztes”: Letztes Element - “summe”: Summe aller vier Elemente

Testen Sie mit `[10, 20, 30, 40]`.

4.4.1.1 Hilfestellungen Problemanalyse

- Wie ermittelt man die Anzahl? → Mit `len(liste)`
- Wie greift man auf das erste Element zu? → `liste[0]`
- Wie greift man auf das letzte Element zu? → `liste[-1]`
- Wie berechnet man die Summe ohne Schleife? → Alle vier Elemente addieren

Lösungsschritte

1. Funktion mit Parameter `zahlen` definieren
2. Anzahl mit `len()` ermitteln
3. Erstes Element mit Index `[0]` holen
4. Letztes Element mit Index `[-1]` holen
5. Summe durch Addition aller vier Elemente berechnen
6. Formatierten String mit f-String erstellen
7. String mit `return` zurückgeben

Pseudocode

```
FUNKTION analysiere_liste(zahlen):  
    anzahl = LÄNGE VON zahlen  
    erstes = zahlen[0]  
    letztes = zahlen[-1]  
    summe = zahlen[0] + zahlen[1] + zahlen[2] + zahlen[3]  
  
    RÜCKGABE f"anzahl: {anzahl}, erstes: {erstes}, letztes: {letztes}, summe: {summe}"  
  
test_liste = [10, 20, 30, 40]  
ergebnis = analysiere_liste(test_liste)  
AUSGABE f"Analyse: {ergebnis}"
```

4.4.2 Erwartete Ausgabe

```
"anzahl: 4, erstes: 10, letztes: 40, summe: 100"
```


5 Übungsblock B – Dictionaries und Funktionen

Aufwand ca. 20 - 30 Minuten

Fortschritt: [#### _____] 40% der Übung

Info-Box: Dictionary-Operationen

Wichtige Dictionary-Operationen für diese Übungen: - `dict["schlüssel"]` - Wert abrufen - `dict["neuer_schlüssel"] = wert` - Neuen Eintrag hinzufügen - `dict.keys()` - Alle Schlüssel als Liste-ähnliches Objekt - `dict.values()` - Alle Werte als Liste-ähnliches Objekt

5.1 Aufgabe 4: Produktpreis berechnen

5.1.1 Aufgabenstellung

Schreiben Sie eine Funktion `berechne_gesamtpreis()`, die ein Produkt-Dictionary mit den Schlüsseln "name", "preis" und "anzahl" erhält und den Gesamtpreis berechnet.

Formel: Gesamtpreis = preis * anzahl

Testen Sie mit:

```
produkt = {"name": "Notebook", "preis": 899.99, "anzahl": 3}
```

5.1.1.1 Hilfestellungen Problemanalyse

- Wie greift man auf Dictionary-Werte zu? → Mit `dict["schlüssel"]`
- Welche Werte werden benötigt? → "name", "preis", "anzahl"
- Was soll berechnet werden? → $\text{preis} \times \text{anzahl}$
- Was soll ausgegeben werden? → Formatierte Ausgabe mit Name, Stückzahl und Gesamtpreis

Lösungsschritte

1. Funktion mit Parameter `produkt` definieren
2. Werte aus Dictionary extrahieren: `produkt["name"]`, `produkt["preis"]`, `produkt["anzahl"]`
3. Gesamtpreis berechnen: `preis * anzahl`
4. Formatierte Ausgabe mit f-String erstellen
5. Gesamtpreis mit `return` zurückgeben

Pseudocode

```
FUNKTION berechne_gesamtpreis(produkt):
    name = produkt["name"]
    preis = produkt["preis"]
    anzahl = produkt["anzahl"]

    gesamtpreis = preis * anzahl

    AUSGABE f"{name} ({anzahl} Stück): {gesamtpreis} Euro"

    RÜCKGABE gesamtpreis

produkt = {"name": "Notebook", "preis": 899.99, "anzahl": 3}
gesamt = berechne_gesamtpreis(produkt)
```

5.1.2 Erwartete Ausgabe

Notebook (3 Stück): 2699.97 Euro

5.2 Aufgabe 5: Studenten-Dictionary erweitern

5.2.1 Aufgabenstellung

Schreiben Sie eine Funktion `berechne_notenschnitt()`, die ein Dictionary mit drei Noten erhält und den Durchschnitt als neuen Schlüssel hinzufügt.

Das Dictionary soll vorher die Schlüssel “mathe”, “physik” und “informatik” haben. Die Funktion fügt “durchschnitt” hinzu und gibt das erweiterte Dictionary zurück.

Testen Sie mit:

```
noten = {"mathe": 1.7, "physik": 2.3, "informatik": 1.3}
```

5.2.1.1 Hilfestellungen Problemanalyse

- Wie liest man Werte aus dem Dictionary? → `dict["schlüssel"]`
- Wie fügt man einen neuen Schlüssel hinzu? → `dict["neuer_schlüssel"] = wert`
- Wie rundet man auf 2 Nachkommastellen? → `round(zahl, 2)`
- Was passiert mit dem Original-Dictionary? → Es wird direkt modifiziert

Lösungsschritte

1. Funktion mit Parameter `noten_dict` definieren
2. Die drei Noten aus dem Dictionary lesen
3. Summe berechnen
4. Durchschnitt berechnen (Summe / 3)
5. Durchschnitt auf 2 Nachkommastellen runden
6. Neuen Schlüssel “durchschnitt” zum Dictionary hinzufügen
7. Erweitertes Dictionary zurückgeben

Pseudocode

```
FUNKTION berechne_notenschnitt(noten_dict):
    mathe = noten_dict["mathe"]
    physik = noten_dict["physik"]
    informatik = noten_dict["informatik"]

    summe = mathe + physik + informatik
    durchschnitt = summe / 3
    durchschnitt = RUNDE(durchschnitt, 2)

    noten_dict["durchschnitt"] = durchschnitt

    RÜCKGABE noten_dict

noten = {"mathe": 1.7, "physik": 2.3, "informatik": 1.3}
ergebnis = berechne_notenschnitt(noten)
AUSGABE f"Noten mit Durchschnitt: {ergebnis}"
```

5.2.2 Erwartete Ausgabe

Noten mit Durchschnitt: {'mathe': 1.7, 'physik': 2.3, 'informatik': 1.3, 'durchschnitt': 1.77}

5.3 Aufgabe 6: Zwei Dictionaries vergleichen

5.3.1 Aufgabenstellung

Schreiben Sie eine Funktion `vergleiche_personen()`, die zwei Personen-Dictionaries erhält (jeweils mit "name" und "alter") und ein neues Dictionary mit Vergleichsinformationen zurückgibt: - "aelter": Name der älteren Person - "juenger": Name der jüngeren Person - "altersunterschied": Differenz in Jahren

Testen Sie mit:

```
person1 = {"name": "Max", "alter": 25}
person2 = {"name": "Lisa", "alter": 30}
```

5.3.1.1 Hilfestellungen Problemanalyse

- Wie vergleicht man zwei Werte? → Mit > Operator
- Wie wählt man ohne if den richtigen Namen? → Boolean-Multiplikation-Trick
- Wie berechnet man den Betrag einer Differenz? → Auch mit Boolean-Trick oder Fallunterscheidung
- Ergebnis-Dictionary: Wie erstellt man ein leeres? → `ergebnis = {}`

Lösungsschritte

1. Funktion mit zwei Parametern `person1` und `person2` definieren
2. Namen und Alter aus beiden Dictionaries extrahieren
3. Vergleich durchführen: `person1_aelter = alter1 > alter2`
4. Mit Boolean-Multiplikation den Namen der älteren/jüngeren Person bestimmen
5. Altersunterschied berechnen (Betrag der Differenz)
6. Ergebnis-Dictionary erstellen und befüllen
7. Dictionary zurückgeben

Pseudocode

```
FUNKTION vergleiche_personen(person1, person2):
    name1 = person1["name"]
    alter1 = person1["alter"]
    name2 = person2["name"]
    alter2 = person2["alter"]

    vergleich = {}

    person1_aelter = alter1 > alter2

    # Boolean-Trick: True * "Max" = "Max", False * "Max" = ""
    vergleich["aelter"] = name1 * person1_aelter + name2 * (NICHT person1_aelter)
    vergleich["juenger"] = name2 * person1_aelter + name1 * (NICHT person1_aelter)

    # Altersunterschied (Betrag)
    unterschied = alter1 - alter2
    vergleich["altersunterschied"] = unterschied * (unterschied >= 0) + (-unterschied) * (unterschied < 0)

    RÜCKGABE vergleich

person1 = {"name": "Max", "alter": 25}
person2 = {"name": "Lisa", "alter": 30}
ergebnis = vergleiche_personen(person1, person2)
AUSGABE f"Vergleich: {ergebnis}"
```

5.3.2 Erwartete Ausgabe

Vergleich: {'aelter': 'Lisa', 'juenger': 'Max', 'altersunterschied': 5}

6 Übungsblock C – Vertiefung Funktionskonzepte

Aufwand ca. 20 - 40 Minuten

Fortschritt: [#####] 60% der Übung

Info-Box: Wichtige Funktionskonzepte - Scope: Der Gültigkeitsbereich von Variablen (lokal vs. global) - **Parameter-Arten:** Positional (Reihenfolge wichtig) vs. Benannt (flexibel) - **Return:** Gibt Werte zurück vs. `print()` zeigt nur an

6.1 Aufgabe 7: Lokale vs. Globale Variablen

Schwierigkeit: ** (Mittel)

6.1.1 Aufgabenstellung

Untersuchen Sie den Unterschied zwischen lokalen und globalen Variablen. Erstellen Sie: 1. Eine globale Variable `punktestand` mit dem Wert 100 2. Eine Funktion `aendere_lokal()`, die eine lokale Variable `punktestand = 200` erstellt und zurückgibt 3. Eine Funktion `zeige_global()`, die die globale Variable zurückgibt 4. Testen Sie beide Funktionen und geben Sie die Ergebnisse aus

6.1.1.1 Hilfestellungen Problemanalyse

- Was ist eine globale Variable? → Außerhalb von Funktionen definiert, überall sichtbar
- Was ist eine lokale Variable? → Innerhalb einer Funktion definiert, nur dort sichtbar
- Was passiert bei gleichem Namen? → Lokale “überdeckt” globale temporär
- Wird die globale Variable verändert? → Nein, lokale ist eine neue, separate Variable

Lösungsschritte

1. Globale Variable `punktestand = 100` definieren (außerhalb von Funktionen)
2. Funktion `aendere_lokal()` erstellen, die lokale Variable `punktestand = 200` setzt
3. Funktion `zeige_global()` erstellen, die `punktestand` zurückgibt
4. Beide Funktionen aufrufen und Ergebnisse speichern
5. Alle drei Werte ausgeben (lokal, global aus Funktion, globale Variable direkt)

Pseudocode

```
# Globale Variable
punktestand = 100

FUNKTION aendere_lokal():
    punktestand = 200 # Dies ist LOKAL, nicht global!
    RÜCKGABE punktestand

FUNKTION zeige_global():
    RÜCKGABE punktestand # Verwendet die globale Variable

# Test
lokal_wert = aendere_lokal()
global_wert = zeige_global()

AUSGABE f"Lokaler Wert aus aendere_lokal(): {lokal_wert}" # 200
AUSGABE f"Globaler Wert aus zeige_global(): {global_wert}" # 100
AUSGABE f"Globale Variable punktestand: {punktestand}" # 100
```

Erwartetes Verhalten: Die lokale Änderung beeinflusst die globale Variable nicht!

6.2 Aufgabe 8: Variable Shadowing

Schwierigkeit: *** (Mittel)

6.2.1 Aufgabenstellung

Demonstrieren Sie “Variable Shadowing” (wenn lokale und globale Variablen denselben Namen haben):

```
counter = 10 # Global
```

```
def funktion_a():
    counter = 20
    return counter
```

```
def funktion_b():
    return counter + 5
```

Rufen Sie beide Funktionen, speichern Sie die Ergebnisse und geben Sie alle drei Werte aus. Erklären Sie das Ergebnis mit Kommentaren in Ihrem Programm.

6.2.1.1 Hilfestellungen Problemanalyse

- Was ist “Shadowing”? → Lokale Variable “verdeckt” gleichnamige globale Variable
- Welche Funktion nutzt die lokale Variable? → `funktion_a()` (erstellt eigene)
- Welche Funktion nutzt die globale Variable? → `funktion_b()` (keine lokale Definition)
- Wird die globale Variable verändert? → Nein

Lösungsschritte

1. Gegebenen Code übernehmen
2. Beide Funktionen aufrufen und Ergebnisse speichern
3. Alle drei Werte ausgeben (Ergebnis a, Ergebnis b, globale Variable)
4. Erklärende Kommentare hinzufügen

Pseudocode

```
counter = 10 # Globale Variable
```

```
FUNKTION funktion_a():
    counter = 20 # Lokale Variable (shadowing)
    RÜCKGABE counter
```

```
FUNKTION funktion_b():
    # Keine lokale Variable, nutzt globale
    RÜCKGABE counter + 5
```

```
# Aufrufe
ergebnis_a = funktion_a() # Gibt 20 zurück (lokale Variable)
ergebnis_b = funktion_b() # Gibt 15 zurück (10 + 5, globale Variable)
```

```
AUSGABE f"funktion_a() gibt zurück: {ergebnis_a}" # 20
AUSGABE f"funktion_b() gibt zurück: {ergebnis_b}" # 15
AUSGABE f"Globale Variable counter: {counter}" # 10 (unverändert)
```

```
# Erklärung als Kommentar:
# funktion_a erstellt eine NEUE lokale Variable namens counter
# funktion_b nutzt die GLOBALE Variable counter
# Die globale Variable wird nie verändert
```

6.3 Aufgabe 9: Scope-Kette

Schwierigkeit: *** (Mittel-Schwer)

6.3.1 Aufgabenstellung

Erstellen Sie eine "Scope-Kette" mit drei Funktionen, die sich gegenseitig aufrufen: 1. `berechne_basis()` - gibt 10 zurück 2. `verdopple_basis()` - ruft `berechne_basis()` auf und verdoppelt das Ergebnis 3. `finale_berechnung()` - ruft `verdopple_basis()` auf und addiert 30

Verfolgen Sie den Datenfluss durch alle Funktionen.

6.3.1.1 Hilfestellungen Problemanalyse

- Wie ruft eine Funktion eine andere auf? → Einfach den Funktionsnamen mit Klammern verwenden
- Wie verwendet man das Ergebnis? → In einer Variable speichern
- Wie verfolgt man den Datenfluss? → print-Ausgaben in jeder Funktion einbauen

Lösungsschritte

1. Funktion `berechne_basis()` erstellen: gibt 10 zurück
2. Funktion `verdopple_basis()` erstellen: ruft `berechne_basis()` auf, verdoppelt das Ergebnis
3. Funktion `finale_berechnung()` erstellen: ruft `verdopple_basis()` auf, addiert 30
4. In jeder Funktion eine print-Ausgabe einbauen zur Nachverfolgung
5. `finale_berechnung()` aufrufen und Endergebnis ausgeben

Pseudocode

```
FUNKTION berechne_basis():
    basis = 10
    AUSGABE f" berechne_basis() gibt {basis} zurück"
    RÜCKGABE basis

FUNKTION verdopple_basis():
    wert = berechne_basis() # Ruft erste Funktion auf
    ergebnis = wert * 2
    AUSGABE f" verdopple_basis() gibt {ergebnis} zurück"
    RÜCKGABE ergebnis

FUNKTION finale_berechnung():
    wert = verdopple_basis() # Ruft zweite Funktion auf
    endergebnis = wert + 30
    AUSGABE f" finale_berechnung() gibt {endergebnis} zurück"
    RÜCKGABE endergebnis

# Datenfluss verfolgen
AUSGABE "Starte Berechnung..."
resultat = finale_berechnung()
AUSGABE f"Finales Ergebnis: {resultat}"

# Datenfluss: 10 -> 20 -> 50
```

6.4 Aufgabe 10: Positional vs. Named Parameters

Schwierigkeit: ** (Leicht-Mittel)

6.4.1 Aufgabenstellung

Erstellen Sie eine Funktion `erstelle_profil()` mit 4 Parametern: - `vorname` - `nachname` - `alter` - `stadt`

Die Funktion soll ein formatiertes Profil als Dictionary zurückgeben.

Rufen Sie die Funktion **dreimal** auf: 1. Nur mit positionalen Argumenten 2. Nur mit benannten Argumenten (in anderer Reihenfolge!) 3. Gemischt: erste zwei positional, letzte zwei benannt

6.4.1.1 Hilfestellungen Problemanalyse

- Was sind positionale Argumente? → Werte in der definierten Reihenfolge übergeben
- Was sind benannte Argumente? → **parameter=wert**, Reihenfolge egal
- Dürfen beide gemischt werden? → Ja, aber positionale MÜSSEN vor benannten kommen

Lösungsschritte

1. Funktion mit 4 Parametern definieren
2. Innerhalb: Dictionary mit "name", "alter", "stadt" erstellen
3. "name" aus Vor- und Nachname zusammensetzen
4. Dictionary zurückgeben
5. Drei verschiedene Aufrufe durchführen und ausgeben

Pseudocode

```
FUNKTION erstelle_profil(vorname, nachname, alter, stadt):
    profil = {
        "name": f"{vorname} {nachname}",
        "alter": alter,
        "stadt": stadt
    }
    RÜCKGABE profil

# 1. Nur positionale Argumente (Reihenfolge wichtig!)
profil1 = erstelle_profil("Max", "Müller", 25, "Berlin")
AUSGABE f"Profil 1: {profil1}"

# 2. Nur benannte Argumente (Reihenfolge egal!)
profil2 = erstelle_profil(
    stadt="Hamburg",
    alter=30,
    nachname="Schmidt",
    vorname="Anna"
)
AUSGABE f"Profil 2: {profil2}"

# 3. Gemischt: erste zwei positional, letzte zwei benannt
profil3 = erstelle_profil("Tom", "Wagner", alter=28, stadt="München")
AUSGABE f"Profil 3: {profil3}"
```

6.4.2 Erwartete Ausgabe

```
Profil 1: {'name': 'Max Müller', 'alter': 25, 'stadt': 'Berlin'}
Profil 2: {'name': 'Anna Schmidt', 'alter': 30, 'stadt': 'Hamburg'}
Profil 3: {'name': 'Tom Wagner', 'alter': 28, 'stadt': 'München'}
```

6.5 Aufgabe 11: Parameter-Reihenfolge verstehen

Schwierigkeit: *** (Mittel)

6.5.1 Aufgabenstellung

Erstellen Sie eine Funktion `berechne_preis()` mit drei Parametern: - `grundpreis` - `rabatt_prozent` - `versandkosten`

Die Funktion berechnet: $(\text{grundpreis} * (1 - \text{rabatt_prozent}/100)) + \text{versandkosten}$

Zeigen Sie, was passiert, wenn die Argumente in falscher Reihenfolge übergeben werden: 1. Korrekt: `berechne_preis(100, 20, 5)` 2. Falsch: `berechne_preis(20, 100, 5)` 3. Mit benannten Argumenten korrigiert

6.5.1.1 Hilfestellungen Problemanalyse

- Was passiert bei falscher Reihenfolge? → Werte werden falschen Parametern zugeordnet
- Warum ist 100% Rabatt unsinnig? → Preis wird 0 (minus Versand)
- Wie verhindert man Fehler? → Benannte Argumente verwenden

Lösungsschritte

1. Funktion mit drei Parametern definieren
2. Formel implementieren: $\text{grundpreis} * (1 - \text{rabatt_prozent}/100) + \text{versandkosten}$
3. Korrekten Aufruf durchführen und Ergebnis ausgeben
4. Falschen Aufruf durchführen und Ergebnis ausgeben
5. Korrektur mit benannten Argumenten durchführen

Pseudocode

```
FUNKTION berechne_preis(grundpreis, rabatt_prozent, versandkosten):
    preis_nach_rabatt = grundpreis * (1 - rabatt_prozent/100)
    endpreis = preis_nach_rabatt + versandkosten
    RÜCKGABE endpreis

# 1. Korrekte Reihenfolge
korrekt = berechne_preis(100, 20, 5)
AUSGABE f"Korrekt (100€, 20% Rabatt, 5€ Versand): {korrekt}€"
# Rechnung: 100 * 0.8 + 5 = 85€

# 2. Falsche Reihenfolge (vertauscht)
falsch = berechne_preis(20, 100, 5)
AUSGABE f"Falsch (20€, 100% Rabatt?, 5€ Versand): {falsch}€"
# Rechnung: 20 * 0 + 5 = 5€ (unsinnig!)

# 3. Mit benannten Argumenten korrigiert
korrigiert = berechne_preis(
    rabatt_prozent=20,
    grundpreis=100,
    versandkosten=5
)
AUSGABE f"Korrigiert mit benannten Argumenten: {korrigiert}€"
```

6.6 Aufgabe 12: Gemischte Parameter-Strategie

Schwierigkeit: *** (Mittel)

6.6.1 Aufgabenstellung

Schreiben Sie eine Funktion `formatiere_nachricht()` die: - Zwei positionale Parameter hat: `empfaenger`, `betreff` - Zwei weitere Parameter hat, die Sie mal positional, mal benannt übergeben: `text`, `prioritaet` - Die Priorität soll standardmäßig `niedrig` sein.

Demonstrieren Sie verschiedene gültige Aufrufvarianten und kommentieren Sie, warum manche funktionieren und andere nicht. Was passiert, wenn Sie keinen Wert für die Priorität vergeben?

6.6.1.1 Hilfestellungen Problemanalyse

- Was ist ein Standardwert? → Wert, der verwendet wird, wenn kein Argument übergeben wird
- Wie definiert man einen Standardwert? → `parameter="standard"` in der Funktionsdefinition
- Was passiert ohne Angabe? → Der Standardwert wird verwendet

Lösungsschritte

1. Funktion mit 4 Parametern definieren, `prioritaet` mit Standardwert
2. Formatierte Nachricht als String erstellen
3. Variante 1: Alle Parameter positional übergeben
4. Variante 2: Gemischt (erste zwei positional, Rest benannt)
5. Variante 3: Alle benannt
6. Variante 4: Ohne Priorität (Standardwert wird genutzt)

Pseudocode

```
FUNKTION formatiere_nachricht(empfaenger, betreff, text, prioritaet="Niedrig"):
    nachricht = f"""
An: {empfaenger}
Betreff: {betreff}
Priorität: {prioritaet}
---
{text}
"""
    RÜCKGABE nachricht

# Variante 1: Alle positional
msg1 = formatiere_nachricht("Max", "Meeting", "Bitte um Rückruf", "Hoch")
AUSGABE "Variante 1 (alle positional):", msg1

# Variante 2: Gemischt
msg2 = formatiere_nachricht("Anna", "Info", prioritaet="Normal", text="FYI")
AUSGABE "Variante 2 (gemischt):", msg2

# Variante 3: Alle benannt
msg3 = formatiere_nachricht(
    text="Dringend!",
    empfaenger="Tom",
    prioritaet="Kritisch",
    betreff="Problem"
)
AUSGABE "Variante 3 (alle benannt):", msg3

# Variante 4: Ohne Priorität (Standardwert)
msg4 = formatiere_nachricht("Lisa", "Update", "Alles okay")
AUSGABE "Variante 4 (Standardwert):", msg4
# -> Priorität ist "Niedrig"
```

6.7 Aufgabe 13: Return vs. Print

Schwierigkeit: ** (Leicht-Mittel)

6.7.1 Aufgabenstellung

Erstellen Sie zwei Versionen derselben Funktion: 1. `multipliziere_print()` - verwendet `print()` zur Ausgabe 2. `multipliziere_return()` - verwendet `return`

Beide nehmen zwei Zahlen und berechnen das Produkt.

Zeigen Sie den praktischen Unterschied: - Versuchen Sie, mit beiden Ergebnissen weiterzurechnen - Was passiert, wenn Sie das "Ergebnis" von `multipliziere_print()` in einer Variable speichern?

6.7.1.1 Hilfestellungen Problemanalyse

- Was macht `print()`? → Zeigt etwas an, gibt aber `None` zurück
- Was macht `return`? → Gibt einen Wert zurück, der weiterverwendet werden kann
- Was passiert bei `ergebnis = funktion_mit_print()`? → `ergebnis` wird `None`
- Kann man mit `None` rechnen? → Nein, führt zu `TypeError`

Lösungsschritte

1. Funktion `multipliziere_print()` erstellen: berechnet und gibt mit `print()` aus
2. Funktion `multipliziere_return()` erstellen: berechnet und gibt mit `return` zurück
3. Beide Funktionen aufrufen und Ergebnisse in Variablen speichern
4. Ausgeben, was in den Variablen steht
5. Versuchen, mit dem Ergebnis der `return`-Version weiterzurechnen
6. Kommentieren, warum `print`-Version nicht weiterverwendet werden kann

Pseudocode

```
FUNKTION multipliziere_print(a, b):
    produkt = a * b
    AUSGABE f"{a} × {b} = {produkt}"
    # Kein return, also gibt die Funktion None zurück!

FUNKTION multipliziere_return(a, b):
    produkt = a * b
    RÜCKGABE produkt

# Test mit print-Version
AUSGABE "Mit print():"
ergebnis_print = multipliziere_print(5, 3) # Zeigt: 5 × 3 = 15
AUSGABE f"Gespeicherter Wert: {ergebnis_print}" # None!
# Weiterrechnen geht nicht:
# weiter = ergebnis_print + 10 # TypeError!

AUSGABE "Mit return:"
ergebnis_return = multipliziere_return(5, 3)
AUSGABE f"Gespeicherter Wert: {ergebnis_return}" # 15

# Weiterrechnen funktioniert:
weiter = ergebnis_return + 10
AUSGABE f"Weitergerechnet: {ergebnis_return} + 10 = {weiter}"
```

6.8 Aufgabe 14: Mehrfache Return-Werte

Schwierigkeit: *** (Mittel)

6.8.1 Aufgabenstellung

Erstellen Sie eine Funktion `analysiere_einkauf()`, die eine Liste von Preisen erhält und zurückgibt: 1. Die Anzahl der Artikel 2. Die Summe aller Preise 3. Den teuersten Artikel 4. Den günstigsten Artikel

Verwenden Sie die Funktion mit der Liste [12.99, 45.50, 3.99, 27.80, 9.99].

Hinweis: Ohne Schleifen! Nutzen Sie `sum()`, `max()`, `min()` und `len()`.

6.8.1.1 Hilfestellungen Problemanalyse

- Wie gibt man mehrere Werte zurück? → Mit Komma getrennt: `return a, b, c, d`
- Wie empfängt man mehrere Werte? → Mit Komma getrennt: `a, b, c, d = funktion()`
- Welche eingebauten Funktionen helfen? → `len()`, `sum()`, `max()`, `min()`

Lösungsschritte

1. Funktion mit Parameter `preisliste` definieren
2. Anzahl mit `len()` ermitteln
3. Summe mit `sum()` berechnen
4. Maximum mit `max()` finden
5. Minimum mit `min()` finden
6. Alle vier Werte mit `return` zurückgeben
7. Beim Aufruf alle vier Werte in Variablen speichern

Pseudocode

```
FUNKTION analysiere_einkauf(preisliste):
    anzahl = LÄNGE VON preisliste
    summe = SUMME VON preisliste
    teuerster = MAXIMUM VON preisliste
    guenstigster = MINIMUM VON preisliste

    RÜCKGABE anzahl, summe, teuerster, guenstigster

# Test
preise = [12.99, 45.50, 3.99, 27.80, 9.99]

# Alle Rückgabewerte empfangen
anzahl, gesamt, max_preis, min_preis = analysiere_einkauf(preise)

AUSGABE f"Anzahl Artikel: {anzahl}"
AUSGABE f"Gesamtpreis: {gesamt}"
AUSGABE f"Teuerster Artikel: {max_preis}"
AUSGABE f"Günstigster Artikel: {min_preis}"
```

6.8.2 Erwartete Ausgabe

```
Anzahl Artikel: 5
Gesamtpreis: 100.27
Teuerster Artikel: 45.50
Günstigster Artikel: 3.99
```

6.9 Aufgabe 15: Return-Kette (Funktionen verketteten)

Schwierigkeit: **** (Fortgeschritten)

6.9.1 Aufgabenstellung

Erstellen Sie eine Kette von Funktionen, die Daten transformieren:

1. `hole_rohwert()` - gibt 42 zurück
2. `konvertiere_zu_euro(wert)` - multipliziert mit 0.85

3. `addiere_steuer(betrag)` - multipliziert mit 1.19
4. `runde_betrag(betrag)` - rundet auf 2 Nachkommastellen

Verketten Sie alle Funktionen in einer Zeile und zeigen Sie auch die Zwischenschritte.

Beispiel-Verkettung:

```
endergebnis = funktion3(funktion2(funktion1(42)))
```

6.9.1.1 Hilfestellungen Problemanalyse

- Wie verkettet man Funktionen? → Das Ergebnis einer Funktion als Parameter der nächsten übergeben
- In welcher Reihenfolge liest man? → Von innen nach außen
- Warum Zwischenschritte zeigen? → Zur Nachvollziehbarkeit des Datenflusses

Lösungsschritte

1. Funktion `hole_rohwert()` erstellen: gibt 42 zurück
2. Funktion `konvertiere_zu_euro(wert)` erstellen: `return wert * 0.85`
3. Funktion `addiere_steuer(betrag)` erstellen: `return betrag * 1.19`
4. Funktion `runde_betrag(betrag)` erstellen: `return round(betrag, 2)`
5. Zwischenschritte einzeln berechnen und ausgeben
6. Alle Funktionen in einer Zeile verketteten

Pseudocode

```
FUNKTION hole_rohwert():
    RÜCKGABE 42
```

```
FUNKTION konvertiere_zu_euro(wert):
    RÜCKGABE wert * 0.85
```

```
FUNKTION addiere_steuer(betrag):
    RÜCKGABE betrag * 1.19
```

```
FUNKTION runde_betrag(betrag):
    RÜCKGABE RUNDE(betrag, 2)
```

```
# Zwischenschritte anzeigen
schritt1 = hole_rohwert()           # 42
AUSGABE f"Schritt 1 - Rohwert: {schritt1}"

schritt2 = konvertiere_zu_euro(schritt1) # 35.7
AUSGABE f"Schritt 2 - In Euro: {schritt2}"

schritt3 = addiere_steuer(schritt2)    # 42.483
AUSGABE f"Schritt 3 - Mit Steuer: {schritt3}"

schritt4 = runde_betrag(schritt3)      # 42.48
AUSGABE f"Schritt 4 - Gerundet: {schritt4}"
```

```
# Alles in einer Zeile
endergebnis = runde_betrag(addiere_steuer(konvertiere_zu_euro(hole_rohwert()))))
AUSGABE f"Endergebnis: {endergebnis}€"
```

7 Erweiterte Aufgabe – Kombination aller Konzepte

Aufwand ca. 20-40 Minuten.

Fortschritt: [#####] 80% der Übung

7.1 Komplexaufgabe: Lagerverwaltungssystem

Schwierigkeit: **** (Mittel-Schwer)

Sie sollen ein einfaches Lagerverwaltungssystem mit Funktionen implementieren. Das System arbeitet mit Listen von Dictionaries.

7.1.1 Teil A: Datenstruktur verstehen

Gegeben ist folgende Struktur für Produkte:

```
produkt = {
    "id": 1,
    "name": "Laptop",
    "bestand": 15,
    "preis": 799.99
}
```

Ein Lager ist eine Liste von Produkten:

```
lager = [
    {"id": 1, "name": "Laptop", "bestand": 15, "preis": 799.99},
    {"id": 2, "name": "Maus", "bestand": 50, "preis": 19.99},
    {"id": 3, "name": "Tastatur", "bestand": 30, "preis": 49.99}
]
```

7.1.2 Teil B: Funktionen implementieren

Aufgabe 16a: Schreiben Sie eine Funktion `berechne_lagerwert()`, die eine Liste mit genau drei Produkten erhält und den Gesamtwert des Lagers berechnet.

Formel: $\text{Wert} = (\text{bestand1} \times \text{preis1}) + (\text{bestand2} \times \text{preis2}) + (\text{bestand3} \times \text{preis3})$

Aufgabe 16b: Schreiben Sie eine Funktion `finde_teuerstes_produkt()`, die aus einer Liste mit drei Produkten das teuerste zurückgibt (basierend auf dem Einzelpreis).

Aufgabe 16c: Schreiben Sie eine Funktion `erstelle_lagerbericht()`, die ein Dictionary mit folgendem Bericht erstellt: - "anzahl_produkte": 3 (fest) - "gesamtwert": Ergebnis von `berechne_lagerwert()` - "teuerstes": Name des teuersten Produkts - "günstigstes": Name des günstigsten Produkts

7.1.3 Erwartete Ausgaben

Lagerwert: 14999.35 Euro

Teuerstes Produkt: Laptop

Lagerbericht: {'anzahl_produkte': 3, 'gesamtwert': 14999.35, 'teuerstes': 'Laptop', 'günstigstes':

7.1.4 Hilfestellungen

7.1.4.1 Aufgabe 16a: Lagerwert berechnen Problemanalyse

- Was ist der Lagerwert? → Summe aller (Bestand × Preis) für jedes Produkt
- Wie greift man auf ein Produkt in der Liste zu? → Mit Index: `lager[0]`, `lager[1]`, `lager[2]`
- Wie greift man auf einen Wert im Dictionary zu? → Mit Schlüssel: `produkt["bestand"]`, `produkt["preis"]`
- Was soll die Funktion zurückgeben? → Den berechneten Gesamtwert (gerundet auf 2 Stellen)

Lösungsschritte

1. Funktion mit Parameter `lager` definieren
2. Erstes Produkt extrahieren: `produkt1 = lager[0]`
3. Wert für Produkt 1 berechnen: `wert1 = produkt1["bestand"] * produkt1["preis"]`
4. Schritte 2-3 für Produkt 2 und 3 wiederholen
5. Alle drei Werte addieren
6. Ergebnis auf 2 Nachkommastellen runden mit `round(gesamtwert, 2)`
7. Gerundeten Wert mit `return` zurückgeben

Pseudocode

```
FUNKTION berechne_lagerwert(lager):
    # Produkte einzeln extrahieren
    produkt1 = lager[0]
    produkt2 = lager[1]
    produkt3 = lager[2]

    # Wert pro Produkt: Bestand × Preis
    wert1 = produkt1["bestand"] * produkt1["preis"]
    wert2 = produkt2["bestand"] * produkt2["preis"]
    wert3 = produkt3["bestand"] * produkt3["preis"]

    # Gesamtwert berechnen
    gesamtwert = wert1 + wert2 + wert3

    RÜCKGABE RUNDE(gesamtwert, 2)

# Test
wert = berechne_lagerwert(lager)
AUSGABE f"Lagerwert: {wert} Euro"
```

7.1.4.2 Aufgabe 16b: Teuerstes Produkt finden Problemanalyse

- Was ist das teuerste Produkt? → Das mit dem höchsten Einzelpreis (nicht Gesamtwert!)
- Wie vergleicht man drei Werte ohne if-Anweisung? → Mit Vergleichsoperatoren und logischen Operatoren (and, or)
- Wie wählt man einen von drei Werten? → Mit dem Boolean-Multiplikations-Trick: `True = 1, False = 0`
- Was soll die Funktion zurückgeben? → Den Namen des teuersten Produkts

Lösungsschritte

1. Funktion mit Parameter `lager` definieren
2. Alle drei Produkte extrahieren
3. Preise paarweise vergleichen:
 - `p1_teurer_p2 = p1["preis"] > p2["preis"]`
 - `p1_teurer_p3 = p1["preis"] > p3["preis"]`
 - `p2_teurer_p3 = p2["preis"] > p3["preis"]`
4. Logische Kombinationen bilden:
 - P1 ist teuerstes, wenn `p1 > p2 AND p1 > p3`
 - P2 ist teuerstes, wenn `p2 > p3 AND nicht P1 teuerstes`
 - P3 ist teuerstes, wenn weder P1 noch P2 teuerstes
5. Mit Boolean-Trick den richtigen Namen auswählen
6. Namen mit `return` zurückgeben

Pseudocode

```
FUNKTION finde_teuerstes_produkt(lager):
    # Produkte extrahieren
    p1 = lager[0]
    p2 = lager[1]
```

```

p3 = lager[2]

# Paarweise Vergleiche
p1_teurer_p2 = p1["preis"] > p2["preis"]
p1_teurer_p3 = p1["preis"] > p3["preis"]
p2_teurer_p3 = p2["preis"] > p3["preis"]

# Wer ist das teuerste?
p1_ist_teuerstes = p1_teurer_p2 AND p1_teurer_p3
p2_ist_teuerstes = p2_teurer_p3 AND (NICHT p1_ist_teuerstes)
p3_ist_teuerstes = NICHT (p1_ist_teuerstes ODER p2_ist_teuerstes)

# Boolean-Trick: True=1, False=0
# Nur einer der drei Faktoren ist 1, die anderen 0
name = (p1["name"] * p1_ist_teuerstes +
        p2["name"] * p2_ist_teuerstes +
        p3["name"] * p3_ist_teuerstes)

RÜCKGABE name

# Test
teuerstes = finde_teuerstes_produkt(lager)
AUSGABE f"Teuerstes Produkt: {teuerstes}"

```

7.1.4.3 Aufgabe 16c: Lagerbericht erstellen Problemanalyse

- Was enthält der Bericht? → Ein Dictionary mit 4 Schlüsseln: anzahl_produkte, gesamtwert, teuerstes, guenstigstes
- Wie nutzt man bereits erstellte Funktionen? → Aufrufen und Ergebnis speichern
- Wie findet man das günstigste Produkt? → Gleiche Logik wie teuerstes, aber mit < statt >
- Was soll die Funktion zurückgeben? → Das Bericht-Dictionary

Lösungsschritte

1. Funktion mit Parameter `lager` definieren
2. Leeres Dictionary `bericht = {}` erstellen
3. `bericht["anzahl_produkte"] = 3` setzen (fest)
4. `berechne_lagerwert(lager)` aufrufen und in `bericht["gesamtwert"]` speichern
5. `finde_teuerstes_produkt(lager)` aufrufen und in `bericht["teuerstes"]` speichern
6. Günstigstes Produkt finden (wie teuerstes, aber mit <):
 - Paarweise Vergleiche mit < statt >
 - Logische Kombinationen bilden
 - Boolean-Trick für Namensauswahl
7. Namen des günstigsten in `bericht["guenstigstes"]` speichern
8. Bericht-Dictionary mit `return` zurückgeben

Pseudocode

```

FUNKTION erstelle_lagerbericht(lager):
    bericht = {}

    # Anzahl ist fest
    bericht["anzahl_produkte"] = 3

    # Gesamtwert mit bereits erstellter Funktion
    bericht["gesamtwert"] = berechne_lagerwert(lager)

    # Teuerstes mit bereits erstellter Funktion
    bericht["teuerstes"] = finde_teuerstes_produkt(lager)

```

```
# Günstigstes finden (ähnliche Logik wie teuerstes)
p1 = lager[0]
p2 = lager[1]
p3 = lager[2]

p1_billiger_p2 = p1["preis"] < p2["preis"]
p1_billiger_p3 = p1["preis"] < p3["preis"]
p2_billiger_p3 = p2["preis"] < p3["preis"]

p1_ist_billigstes = p1_billiger_p2 AND p1_billiger_p3
p2_ist_billigstes = p2_billiger_p3 AND (NICHT p1_ist_billigstes)
p3_ist_billigstes = NICHT (p1_ist_billigstes ODER p2_ist_billigstes)

bericht["guenstigstes"] = (p1["name"] * p1_ist_billigstes +
                           p2["name"] * p2_ist_billigstes +
                           p3["name"] * p3_ist_billigstes)

RÜCKGABE bericht

# Test
bericht = erstelle_lagerbericht(lager)
AUSGABE f"Lagerbericht: {bericht}"
```


8 Musterlösungen

8.1 Lösungen Block A – Listen und Funktionen

Lösung Aufgabe 1: Durchschnitt berechnen

```
def berechne_durchschnitt_drei(zahlen):  
    """Berechnet den Durchschnitt von drei Zahlen"""  
    # Summe der drei Elemente berechnen  
    summe = zahlen[0] + zahlen[1] + zahlen[2]  
  
    # Durch 3 teilen für Durchschnitt  
    durchschnitt = summe / 3  
  
    return durchschnitt
```

```
# Test  
test_zahlen = [15, 20, 25]  
ergebnis = berechne_durchschnitt_drei(test_zahlen)  
print(f"Der Durchschnitt ist: {ergebnis}")
```

Erklärung: - Wir greifen mit Indizes 0, 1, 2 auf die drei Elemente zu - Die Summe wird durch 3 geteilt (arithmetischer Operator /) - Das Ergebnis wird mit return zurückgegeben

Lösung Aufgabe 2: Maximum finden

```
def finde_maximum_drei(zahlen):  
    """Findet das Maximum aus zwei Zahlen"""  
  
    a = zahlen[0]  
    b = zahlen[1]  
  
    a_groößer_b = a > b  
    b_groößer_a = b > a  
  
    maximum = a * a_groößer_b + b * b_groößer_a  
  
    return maximum  
  
# Test  
test_zahlen = [45, 89]  
ergebnis = finde_maximum_drei_einfach(test_zahlen)  
print(f"Das Maximum ist: {ergebnis}")
```

Erklärung: - True wird zu 1, False zu 0 in Berechnungen - So können wir mit Multiplikation das richtige Element auswählen

Lösung Aufgabe 3: Liste analysieren

```
def analysiere_liste(zahlen):  
    """Analysiert eine Liste mit vier Zahlen"""  
  
    # Anzahl mit len() ermitteln  
    anzahl = len(zahlen)  
  
    # Erstes Element (Index 0)  
    erstes = zahlen[0]  
  
    # Letztes Element (Index -1)
```

```
    letztes = zahlen[-1]

    # Summe aller vier Elemente
    summe = zahlen[0] + zahlen[1] + zahlen[2] + zahlen[3]

    return f"anzahl: {anzahl}, erstes: {erstes}, letztes: {letztes}, summe: {summe}"

# Test
test_liste = [10, 20, 30, 40]
ergebnis = analysiere_liste(test_liste)
print(f"Analyse: {ergebnis}")
```

Erklärung: - len() gibt die Anzahl der Elemente - Index -1 greift auf das letzte Element zu - Die Summe wird durch Addition aller vier Elemente berechnet - f-String wird verwendet, um einen gut lesbaren String zurückzugeben

8.2 Lösungen Block B – Dictionaries und Funktionen

Lösung Aufgabe 4: Produktpreis berechnen

```
def berechne_gesamtpreis(produkt):
    """Berechnet den Gesamtpreis eines Produkts"""
    # Werte aus Dictionary extrahieren
    name = produkt["name"]
    preis = produkt["preis"]
    anzahl = produkt["anzahl"]

    # Gesamtpreis berechnen
    gesamtpreis = preis * anzahl

    # Formatierte Ausgabe
    print(f"{name} ({anzahl} Stück): {gesamtpreis} Euro")

    return gesamtpreis

# Test
produkt = {"name": "Notebook", "preis": 899.99, "anzahl": 3}
gesamt = berechne_gesamtpreis(produkt)
```

Erklärung: - Dictionary-Zugriff mit eckigen Klammern und Schlüssel als String - Multiplikation für Gesamtpreis-Berechnung - F-String für formatierte Ausgabe

Lösung Aufgabe 5: Notenschnitt berechnen

```
def berechne_notenschnitt(noten_dict):
    """Berechnet den Notenschnitt und fügt ihn zum Dictionary hinzu"""
    # Noten extrahieren
    mathe = noten_dict["mathe"]
    physik = noten_dict["physik"]
    informatik = noten_dict["informatik"]

    # Durchschnitt berechnen
    summe = mathe + physik + informatik
    durchschnitt = summe / 3

    # Durchschnitt auf 2 Nachkommastellen runden
    durchschnitt = round(durchschnitt, 2)

    # Zum Dictionary hinzufügen
    noten_dict["durchschnitt"] = durchschnitt

    return noten_dict

# Test
noten = {"mathe": 1.7, "physik": 2.3, "informatik": 1.3}
ergebnis = berechne_notenschnitt(noten)
print(f"Noten mit Durchschnitt: {ergebnis}")
```

Erklärung: - Drei Werte werden aus dem Dictionary gelesen - Durchschnitt wird berechnet (arithmetische Operatoren) - round() rundet auf 2 Nachkommastellen - Neuer Schlüssel wird zum Dictionary hinzugefügt

Lösung Aufgabe 6: Personen vergleichen

```
def vergleiche_personen(person1, person2):
    """Vergleicht zwei Personen nach Alter"""
```

```

# Daten extrahieren
name1 = person1["name"]
alter1 = person1["alter"]
name2 = person2["name"]
alter2 = person2["alter"]

# Ergebnis-Dictionary erstellen
vergleich = {}

# Vergleich mit logischen Operatoren
person1_aelter = alter1 > alter2

# Namen zuordnen basierend auf Vergleich
# Nutze Trick: True = 1, False = 0
vergleich["aelter"] = name1 * person1_aelter + name2 * (not person1_aelter)
vergleich["juenger"] = name2 * person1_aelter + name1 * (not person1_aelter)

# Altersunterschied berechnen (Betrag der Differenz)
unterschied = alter1 - alter2
# Betrag ohne abs() (noch nicht eingeführt)
vergleich["altersunterschied"] = unterschied * (unterschied >= 0) + (-unterschied) * (unterschied < 0)

return vergleich

# Test
person1 = {"name": "Max", "alter": 25}
person2 = {"name": "Lisa", "alter": 30}
ergebnis = vergleiche_personen(person1, person2)
print(f"Vergleich: {ergebnis}")

```

Erklärung: - Vergleichsoperator > prüft, wer älter ist - Logischer Operator not kehrt Bedingung um
 - String-Multiplikation mit True/False für Auswahl - Betrag wird ohne abs() durch Fallunterscheidung berechnet

8.3 Lösungen Block C – Vertiefung Funktionskonzepte

Lösung Aufgabe 7: Lokale vs. Globale Variablen

```

# Globale Variable
punktstand = 100

def aendere_lokal():
    """Erstellt eine lokale Variable mit gleichem Namen"""
    punktstand = 200 # Dies ist LOKAL, nicht global!
    return punktstand

def zeige_global():
    """Gibt die globale Variable zurück"""
    return punktstand

# Test der Funktionen
lokal_wert = aendere_lokal()
global_wert = zeige_global()

print(f"Lokaler Wert aus aendere_lokal(): {lokal_wert}") # 200
print(f"Globaler Wert aus zeige_global(): {global_wert}") # 100
print(f"Globale Variable punktstand: {punktstand}") # 100

# Die globale Variable wurde NICHT verändert!

```

Erklärung: - Die lokale Variable `punktstand` in `aendere_lokal()` ist eine neue Variable - Sie "überdeckt" nur temporär den globalen Namen innerhalb der Funktion - Die globale Variable bleibt unverändert

Lösung Aufgabe 8: Variable Shadowing

```
counter = 10 # Globale Variable

def funktion_a():
    counter = 20 # Lokale Variable (shadowing)
    return counter

def funktion_b():
    # Verwendet die globale Variable
    return counter + 5

# Aufrufe und Ergebnisse
ergebnis_a = funktion_a() # Gibt 20 zurück (lokale Variable)
ergebnis_b = funktion_b() # Gibt 15 zurück (10 + 5, globale Variable)

print(f"funktion_a() gibt zurück: {ergebnis_a}") # 20
print(f"funktion_b() gibt zurück: {ergebnis_b}") # 15
print(f"Globale Variable counter: {counter}")    # 10 (unverändert)

# Erklärung:
# funktion_a erstellt eine NEUE lokale Variable namens counter
# funktion_b nutzt die GLOBALE Variable counter
# Die globale Variable wird nie verändert
```

Wichtig: Variable Shadowing kann zu Verwirrung führen - vermeiden Sie gleiche Namen!

Lösung Aufgabe 9: Scope-Kette

```
def berechne_basis():
    """Gibt den Basiswert zurück"""
    basis = 10
    print(f" berechne_basis() gibt {basis} zurück")
    return basis

def verdopple_basis():
    """Holt Basis und verdoppelt sie"""
    wert = berechne_basis()
    ergebnis = wert * 2
    print(f" verdopple_basis() gibt {ergebnis} zurück")
    return ergebnis

def finale_berechnung():
    """Holt verdoppelte Basis und addiert 30"""
    wert = verdopple_basis()
    endergebnis = wert + 30
    print(f" finale_berechnung() gibt {endergebnis} zurück")
    return endergebnis

# Datenfluss verfolgen
print("Starte Berechnung...")
resultat = finale_berechnung()
print(f"Finales Ergebnis: {resultat}")

# Ausgabe zeigt den kompletten Datenfluss:
```

```
# berechne_basis() -> 10
# verdopple_basis() -> 20
# finale_berechnung() -> 50
```

Datenfluss: 10 -> 20 -> 50

Lösung Aufgabe 10: Positional vs. Named Parameters

```
def erstelle_profil(vorname, nachname, alter, stadt):
    """Erstellt ein Profil-Dictionary"""
    profil = {
        "name": f"{vorname} {nachname}",
        "alter": alter,
        "stadt": stadt
    }
    return profil

# 1. Nur positionale Argumente (Reihenfolge wichtig!)
profil1 = erstelle_profil("Max", "Müller", 25, "Berlin")
print(f"Profil 1: {profil1}")

# 2. Nur benannte Argumente (Reihenfolge egal!)
profil2 = erstelle_profil(
    stadt="Hamburg",
    alter=30,
    nachname="Schmidt",
    vorname="Anna"
)
print(f"Profil 2: {profil2}")

# 3. Gemischt: erste zwei positional, letzte zwei benannt
profil3 = erstelle_profil("Tom", "Wagner", alter=28, stadt="München")
print(f"Profil 3: {profil3}")
```

Merke: Positionale Argumente müssen VOR benannten Argumenten stehen!

Lösung Aufgabe 11: Parameter-Reihenfolge verstehen

```
def berechne_preis(grundpreis, rabatt_prozent, versandkosten):
    """Berechnet Endpreis mit Rabatt und Versand"""
    preis_nach_rabatt = grundpreis * (1 - rabatt_prozent/100)
    endpreis = preis_nach_rabatt + versandkosten
    return endpreis

# 1. Korrekte Reihenfolge
korrekt = berechne_preis(100, 20, 5)
print(f"Korrekt (100€, 20% Rabatt, 5€ Versand): {korrekt}€")
# Rechnung: 100 * 0.8 + 5 = 85€

# 2. Falsche Reihenfolge (vertauscht)
falsch = berechne_preis(20, 100, 5)
print(f"Falsch (20€, 100% Rabatt?, 5€ Versand): {falsch}€")
# Rechnung: 20 * 0 + 5 = 5€ (unsinnig!)

# 3. Mit benannten Argumenten korrigiert
korrigiert = berechne_preis(
    rabatt_prozent=20,
    grundpreis=100,
```

```

    versandkosten=5
)
print(f"Korrigiert mit benannten Argumenten: {korrigiert}€")

```

Lektion: Benannte Argumente verhindern Fehler durch falsche Reihenfolge!

Lösung Aufgabe 12: Gemischte Parameter-Strategie

```

def formatiere_nachricht(empfaenger, betreff, text, prioritaet="Niederig"):
    """Formatiert eine Nachricht"""
    nachricht = f"""
An: {empfaenger}
Betreff: {betreff}
Priorität: {prioritaet}
---
{text}
"""
    return nachricht

# Variante 1: Alle positional (funktioniert)
msg1 = formatiere_nachricht("Max", "Meeting", "Bitte um Rückruf")
print("Variante 1 (alle positional):", msg1)

# Variante 2: Gemischt - erste zwei positional, Rest benannt (funktioniert)
msg2 = formatiere_nachricht("Anna", "Info", prioritaet="Normal", text="FYI")
print("Variante 2 (gemischt):", msg2)

# Variante 3: Alle benannt (funktioniert)
msg3 = formatiere_nachricht(
    text="Dringend!",
    empfaenger="Tom",
    prioritaet="Kritisch",
    betreff="Problem"
)
print("Variante 3 (alle benannt):", msg3)

# FALSCH wäre: benannte vor positionalen Argumenten
# msg_falsch = formatiere_nachricht(empfaenger="Max", "Betreff", "Text", "Normal")
# Das würde einen SyntaxError verursachen!

```

Regel: Positionale Argumente IMMER vor benannten!

Lösung Aufgabe 13: Return vs. Print

```

def multipliziere_print(a, b):
    """Gibt das Produkt nur aus (print)"""
    produkt = a * b
    print(f"{a} × {b} = {produkt}")
    # Kein return, also gibt die Funktion None zurück!

def multipliziere_return(a, b):
    """Gibt das Produkt zurück (return)"""
    produkt = a * b
    return produkt

# Test mit print-Version
print("Mit print():")
ergebnis_print = multipliziere_print(5, 3) # Zeigt: 5 × 3 = 15

```

```

print(f"Gespeicherter Wert: {ergebnis_print}") # None!

# Weiterrechnen geht nicht:
# weiter = ergebnis_print + 10 # TypeError: None + 10 geht nicht!

print("\nMit return:")
ergebnis_return = multipliziere_return(5, 3) # Gibt 15 zurück
print(f"Gespeicherter Wert: {ergebnis_return}") # 15

# Weiterrechnen funktioniert:
weiter = ergebnis_return + 10
print(f"Weitergerechnet: {ergebnis_return} + 10 = {weiter}")

```

Merke: - `print()` zeigt nur an, gibt aber `None` zurück - `return` gibt einen Wert zurück, den Sie weiterverwenden können

Lösung Aufgabe 14: Mehrfache Return-Werte

```

def analysiere_einkauf(preisliste):
    """Analysiert eine Liste von Preisen"""
    anzahl = len(preisliste)
    summe = sum(preisliste)
    teuerster = max(preisliste)
    guenstigster = min(preisliste)

    return anzahl, summe, teuerster, guenstigster

# Test mit der vorgegebenen Liste
preise = [12.99, 45.50, 3.99, 27.80, 9.99]

# Alle Rückgabewerte empfangen
anzahl, gesamt, max_preis, min_preis = analysiere_einkauf(preise)

print(f"Anzahl Artikel: {anzahl}")
print(f"Gesamtpreis: {gesamt}")
print(f"Teuerster Artikel: {max_preis}")
print(f"Günstigster Artikel: {min_preis}")

```

Tipp: Python packt mehrere Return-Werte automatisch in ein Tupel!

Lösung Aufgabe 15: Return-Kette

```

def hole_rohwert():
    """Gibt den Ausgangswert zurück"""
    return 42

def konvertiere_zu_euro(wert):
    """Konvertiert den Wert zu Euro (× 0.85)"""
    return wert * 0.85

def addiere_steuer(betrag):
    """Addiert 19% Mehrwertsteuer"""
    return betrag * 1.19

def runde_betrag(betrag):
    """Rundet auf 2 Nachkommastellen"""
    return round(betrag, 2)

```



```
# Zwischenschritte anzeigen
schritt1 = hole_rohwert()
print(f"Schritt 1 - Rohwert: {schritt1}")

schritt2 = konvertiere_zu_euro(schritt1)
print(f"Schritt 2 - In Euro: {schritt2}")

schritt3 = addiere_steuer(schritt2)
print(f"Schritt 3 - Mit Steuer: {schritt3}")

schritt4 = runde_betrag(schritt3)
print(f"Schritt 4 - Gerundet: {schritt4}")

print("\nDirekt verkettet:")
# Alles in einer Zeile
endergebnis = runde_betrag(addiere_steuer(konvertiere_zu_euro(hole_rohwert()))))

print(f"Endergebnis: {endergebnis}€")

# Alternative: Von innen nach außen lesen
# 1. hole_rohwert() -> 42
# 2. konvertiere_zu_euro(42) -> 35.7
# 3. addiere_steuer(35.7) -> 42.483
# 4. runde_betrag(42.483) -> 42.48
```

Lesehilfe: Bei verketteten Funktionen von innen nach außen lesen!

8.4 Lösung Erweiterte Aufgabe

Lösung Komplexaufgabe: Lagerverwaltung

```
# Aufgabe 16a: Lagerwert berechnen
def berechne_lagerwert(lager):
    """Berechnet den Gesamtwert des Lagers"""
    # Produkte einzeln extrahieren
    produkt1 = lager[0]
    produkt2 = lager[1]
    produkt3 = lager[2]

    # Wert pro Produkt berechnen
    wert1 = produkt1["bestand"] * produkt1["preis"]
    wert2 = produkt2["bestand"] * produkt2["preis"]
    wert3 = produkt3["bestand"] * produkt3["preis"]

    # Gesamtwert
    gesamtwert = wert1 + wert2 + wert3

    return round(gesamtwert, 2)

# Aufgabe 16b: Teuerstes Produkt finden
def finde_teuerstes_produkt(lager):
    """Findet das Produkt mit dem höchsten Einzelpreis"""
    # Produkte extrahieren
    p1 = lager[0]
    p2 = lager[1]
    p3 = lager[2]

    # Preise vergleichen
    p1_teurer_p2 = p1["preis"] > p2["preis"]
    p1_teurer_p3 = p1["preis"] > p3["preis"]
    p2_teurer_p3 = p2["preis"] > p3["preis"]

    # Logik für teuerstes Produkt
    # Wenn p1 teurer als p2 UND p3, dann p1
    p1_ist_teuerstes = p1_teurer_p2 and p1_teurer_p3
    # Wenn p2 teurer als p3 UND nicht p1 teuerstes, dann p2
    p2_ist_teuerstes = p2_teurer_p3 and (not p1_ist_teuerstes)
    # Sonst p3
    p3_ist_teuerstes = not (p1_ist_teuerstes or p2_ist_teuerstes)

    # Produkt zurückgeben basierend auf Bedingung
    # Nutze wieder True=1, False=0 Trick
    teuerstes = {}
    # Einer der drei Werte wird mit 1 multipliziert, die anderen mit 0
    teuerstes["name"] = (p1["name"] * p1_ist_teuerstes +
                          p2["name"] * p2_ist_teuerstes +
                          p3["name"] * p3_ist_teuerstes)

    return teuerstes["name"]

# Aufgabe 16c: Lagerbericht erstellen
def erstelle_lagerbericht(lager):
    """Erstellt einen umfassenden Lagerbericht"""
    bericht = {}

    # Anzahl Produkte (fest 3)
```

```

bericht["anzahl_produkte"] = 3

# Gesamtwert mit vorheriger Funktion
bericht["gesamtwert"] = berechne_lagerwert(lager)

# Teuerstes Produkt mit vorheriger Funktion
bericht["teuerstes"] = finde_teuerstes_produkt(lager)

# Günstigstes Produkt finden (ähnliche Logik)
p1 = lager[0]
p2 = lager[1]
p3 = lager[2]

p1_billiger_p2 = p1["preis"] < p2["preis"]
p1_billiger_p3 = p1["preis"] < p3["preis"]
p2_billiger_p3 = p2["preis"] < p3["preis"]

p1_ist_billigstes = p1_billiger_p2 and p1_billiger_p3
p2_ist_billigstes = p2_billiger_p3 and (not p1_ist_billigstes)
p3_ist_billigstes = not (p1_ist_billigstes or p2_ist_billigstes)

bericht["guenstigstes"] = (p1["name"] * p1_ist_billigstes +
                           p2["name"] * p2_ist_billigstes +
                           p3["name"] * p3_ist_billigstes)

return bericht

# Test mit Beispieldaten
lager = [
    {"id": 1, "name": "Laptop", "bestand": 15, "preis": 799.99},
    {"id": 2, "name": "Maus", "bestand": 50, "preis": 19.99},
    {"id": 3, "name": "Tastatur", "bestand": 30, "preis": 49.99}
]

# Tests durchführen
wert = berechne_lagerwert(lager)
print(f"Lagerwert: {wert} Euro")

teuerstes = finde_teuerstes_produkt(lager)
print(f"Teuerstes Produkt: {teuerstes}")

bericht = erstelle_lagerbericht(lager)
print(f"Lagerbericht: {bericht}")

```

Erklärung der Lösung: - Lagerwert: Jedes Produkt wird einzeln verarbeitet (ohne Schleife) - **Teuerstes finden:** Komplexe Logik mit and/or Operatoren - **Trick:** True=1, False=0 für Auswahl ohne if-Anweisung - **Funktionskomposition:** Funktionen rufen andere Funktionen auf

Häufige Fehler: - Versuch, eine Schleife zu verwenden (noch nicht erlaubt) - Vergessen, dass True/False als 1/0 verwendet werden können - Dictionary-Schlüssel falsch geschrieben

9 Typische Anfängerfehler – Troubleshooting

Fehler	Ursache	Lösung
<code>KeyError: 'schlüssel'</code>	Dictionary-Schlüssel existiert nicht	Schreibweise prüfen, Groß-/Kleinschreibung beachten
<code>IndexError: list index out of range</code>	Zugriff auf nicht existierenden Index	Listenlänge mit <code>len()</code> prüfen
<code>TypeError: unsupported operand</code>	Falsche Datentypen bei Operatoren	Typen prüfen, ggf. konvertieren
<code>NameError: name 'x' is not defined</code>	Variable/Funktion nicht definiert	Schreibweise prüfen, Definition vor Nutzung
Funktion gibt <code>None</code> zurück	<code>return</code> vergessen	<code>return</code> Statement hinzufügen
Logische Operatoren funktionieren nicht	Klammern vergessen	Klammern um Teilbedingungen setzen

10 Abschluss & Reflexion (ca. 10 Min)

10.1 Was Sie gelernt haben

In dieser Übung haben Sie: 1. **Funktionen** zur Verarbeitung von Datenstrukturen geschrieben 2. **Listen und Dictionaries** als Parameter und Rückgabewerte verwendet 3. **Arithmetische und logische Operatoren** in praktischen Szenarien angewendet 4. **Komplexe Probleme** in kleinere Funktionen zerlegt

10.2 Reflexionsfragen

Nehmen Sie sich 5 Minuten Zeit und beantworten Sie für sich:

1. **Welche Aufgabe fanden Sie am schwierigsten?** Warum?
2. **Wo hätten Schleifen geholfen?** Notieren Sie sich Stellen, wo Wiederholungen mühsam waren.
3. **Welche Operator-Kombinationen** waren besonders nützlich?
4. **Wie würden Sie die Funktionen verbessern**, wenn Sie alle Python-Konzepte zur Verfügung hätten?

10.3 Ausblick

Im nächsten Schritt werden Sie lernen: - **if-else-Anweisungen** für echte Entscheidungen (Notebook 09) - **Schleifen** für elegante Wiederholungen (Notebook 10) - **Fehlerbehandlung** für robuste Programme (Notebook 11)

Mit diesen Konzepten werden viele der heutigen Aufgaben deutlich einfacher!

10.4 Selbsteinschätzung

Bewerten Sie Ihr Verständnis:

Konzept	* Unsicher	** Geht so	*** Sicher
Listen-Funktionen			
Dictionary-Funktionen			
Funktionen kombinieren			
Operatoren anwenden			

Empfehlung: Falls Sie bei einem Konzept unsicher sind, wiederholen Sie die entsprechenden Aufgaben oder schauen Sie in die Notebooks 06-08.

Ende der Übungseinheit

Speichern Sie Ihre Lösungen und vergleichen Sie sie mit den Musterlösungen. Bei Fragen wenden Sie sich an Ihre Übungsleitung.

Viel Erfolg beim weiteren Lernen!